

Relazione del Progetto: SailUP

Corso di Tecnologie Web, A.A. 2025-2026



Componenti del gruppo:

Davide Biasuzzi	2111000
Francesco Marcon	2101070
Alberto Reginato	2110450

Referente del gruppo:

davide.biasuzzi@studenti.unipd.it

Indirizzo del sito:

tecweb.studenti.math.unipd.it/dbiasuzz/public/php/index.php

Informazioni di accesso:

- **Amministratore:** username `admin` , password `admin`
- **Utente Semplice:** username `user` , password `user`

Disclaimer: Gli account di test (user, admin), avendo credenziali semplificate per agevolare la fase di revisione, non soddisfano i requisiti della logica di validazione interna che impone l'uso di indirizzi email standard. Di conseguenza, sebbene le funzionalità di autenticazione siano garantite per questi profili, le operazioni di modifica dei dati utente non sono possibili a causa del fallimento della validazione sintattica sui campi. La corretta funzionalità del modulo di gestione profilo è assicurata non appena viene associata una stringa conforme al pattern email previsto dal sistema.

Indice

1	Analisi	3
1.1	Utenza Target	3
1.2	Devices	4
1.3	Funzionalità	4
1.4	Ricerche da soddisfare	5
2	Progettazione	5
2.1	Linee Guida	5
2.2	Struttura	5
3	Realizzazione	6
3.1	Front-End	7
3.1.1	Struttura (HTML)	7
3.1.2	Presentazione (CSS)	7
3.1.2.1	Style.css (Desktop e Base)	8
3.1.2.2	Mobile.css (Dispositivi Portatili)	8
3.1.2.3	Print.css (Stampa)	8
3.1.3	Convenzioni interne	9
3.1.4	Comportamento (JavaScript)	9
3.2	Back-End	10
3.2.1	Architettura (PHP)	10
3.2.1.1	Modularità del codice	11
3.2.2	Logica di Business	11
3.2.2.1	Sistema CRUD Amministrativo	12
3.2.2.2	Gestione Race Condition	12
3.2.2.3	Validazione	13
3.2.3	Sessioni, Sicurezza e Protezione dei Dati	13
3.2.4	Sicurezza lato Server	13
3.2.5	Database (SQL)	14
3.2.5.1	Entità del Database	15
4	Accessibilità e Testing	15
4.1	Accessibilità	16
4.2	Validazione e Testing	17
4.2.1	Validazione	17
4.2.2	Test	18
5	Suddivisione del Lavoro	19
6	Conclusioni e Sviluppi Futuri	20
6.1	Possibili Sviluppi Futuri	20

Abstract

SailUP è una piattaforma web dedicata al noleggio di imbarcazioni e alla prenotazione di esperienze nautiche nel suggestivo scenario del Golfo di Napoli. Siamo onesti, a chi verrebbe l'idea di sviluppare un sito per il corso di Tecnologie Web su un tema così "di nicchia" o, forse qualcuno penserà, così noioso?

Uno dei maggiori problemi riguardanti la gestione tradizionale delle richieste di noleggio e dei tour turistici è la frammentazione dei servizi tra canali telefonici e fisici, che rischia di generare inefficienze e sovrapposizioni nelle disponibilità. SailUP nasce proprio con l'ambizione di mettere ordine in questo *mare magnum*, offrendo un supporto tecnologico capace di ottimizzare i processi gestionali interni e, contemporaneamente, garantire agli utenti un servizio immediato, intuitivo e finalmente autonomo. Il sito web desiderato è dunque focalizzato sull'implementazione delle funzionalità di **Noleggio** e **Esperienze**, guidando l'utente dalla consultazione del catalogo fino alla prenotazione. SailUP sfrutta la piattaforma anche come una vetrina virtuale attraverso una sezione **Blog** dedicata, attraverso cui mira a fornire contenuti informativi di valore ai clienti e a supportare l'attività di content marketing per attrarre nuovi utenti, siano essi turisti o residenti locali. Per chi decide di salire a bordo la piattaforma offre un'**area personale** per la gestione del profilo e il monitoraggio dello storico prenotazioni. Parallelamente, un pannello di **controllo amministrativo** permette la gestione completa dei contenuti dinamici del sito.

1 Analisi

1.1 Utente Target

Per evitare di far naufragare il progetto ancor prima di terminarlo, è fondamentale analizzare l'utente a cui ci rivolgiamo cercando di capire chi, realisticamente, finirebbe per navigare tra le nostre pagine. Senza girarci intorno il noleggio nautico non è esattamente un servizio per le masse. Il target di riferimento si colloca in una fascia di mercato medio-alta, in linea con la natura dei servizi proposti. Ci rivolgiamo quindi ad un'utente con una spiccata capacità di spesa. Se il servizio è premium, l'interfaccia non può permettersi di essere grossolana.

L'utente prevista è eterogenea ma, evitando di perderci in astratte profilazioni marketing, possiamo identificare principalmente due categorie di utenti:

- **Clienti Abituati / Utenti Registrati:** Sono quelli che sanno già cosa vogliono e, probabilmente, hanno poca pazienza. La loro priorità è l'efficienza, vogliono entrare e gestire una prenotazione nel minor tempo possibile. Per loro il sistema deve essere un meccanismo oliato, con accesso rapido ed una semplice gestione del profilo.
- **Nuovi Clienti / Turisti:** Spesso approdano sul sito per caso o per una ricerca Google fatta all'ultimo minuto. Non conoscono il brand e, ammettiamolo, la loro attenzione è una risorsa scarsa. In questo caso la sfida è doppia: dobbiamo prima farci trovare ottimizzando il ranking (la seconda pagina di Google è il posto migliore dove nascondere un cadavere) e poi dobbiamo convincerli a restare. Qui entrano in gioco l'estetica e l'intuitività.

1.2 Devices

Realisticamente, l'utente tipo di SailUP si trova su un molo, sotto il sole, con una connessione instabile e un pollice solo a disposizione. Per questo motivo la priorità è stata adottare un approccio **mobile-first**: l'interfaccia deve essere leggera, i bottoni facili da cliccare anche con le dita bagnate e le informazioni essenziali subito visibili anche con il sole riflesso sullo schermo. Tutto questo garantendo un'ottima fruibilità anche per un classico utilizzo da pc.

1.3 Funzionalità

Sono state individuate e realizzate le seguenti funzionalità principali:

- **Catalogo noleggio, esperienze e Blog:**

- La Home e le pagine di catalogo devono presentare le imbarcazioni e le esperienze con immagini accattivanti e dettagli tecnici chiari.
- È prevista una sezione Blog per il content marketing, necessario ad attirare traffico organico e a fornire consigli (es. itinerari, guide) che trasformino il visitatore curioso in un cliente pagante.
- La grafica deve essere coerente con il tema nautico e garantire la massima leggibilità ed intuitività.

- **Prenotazione:**

- La prenotazione permette di scegliere date, numero di ospiti e servizi extra (es. skipper, per chi preferisce non finire sugli scogli).
- Il sistema deve verificare la disponibilità della risorsa (barca o esperienza) per la data richiesta tramite controlli *server-side* per evitare *overbooking*.
- Le prenotazioni possono essere effettuate da utenti registrati; gli ospiti vengono invitati a registrarsi o accedere per finalizzare la prenotazione.
- In seguito alla richiesta, il sistema fornisce un feedback immediato sull'esito (conferma o errore in caso di date non disponibili).

- **Area Personale e Amministrazione:**

- Sezione **Cliente** per la gestione del profilo e visualizzazione dello storico prenotazioni.
- Sezione **Admin** per il controllo completo della piattaforma. L'amministratore necessita di una visione globale su utenti iscritti e prenotazioni effettuate e un sistema CRUD (Create, Read, Update, Delete) per gestire dinamicamente i prodotti, quindi barche, esperienze ed articoli del blog.

- **Ospite e Registrazione:**

- L'utente non autenticato potrà visualizzare i cataloghi di noleggio barche ed esperienze, leggere gli articoli pubblicati sul blog e accedere alle pagine informative statiche (Chi Siamo, FAQ, Privacy/Cookie Policy). La registrazione avviene tramite un form di compilazione dati per consentire ai nuovi utenti di creare un account, requisito necessario per effettuare una prenotazione e per gestirla.

1.4 Ricerche da soddisfare

Siamo consapevoli che un sito web senza visitatori è solo un esercizio di stile archiviato in un server, o peggio, in un hard-disk. Al fine di migliorare la SEO ed intercettare il target di riferimento abbiamo ottimizzato la struttura per rispondere a intenzioni di ricerca concrete:

- Noleggio barche Golfo di Napoli
- Escursioni in barca Capri e Positano
- Esperienze nautiche Napoli
- Affitto gommone Ischia
- Blog consigli nautica
- Tour in barca con skipper

2 Progettazione

2.1 Linee Guida

Per la gestione del ciclo di vita del software e il coordinamento del team, si è scelto di utilizzare un repository su GitHub per il versionamento del codice.

L'eterogeneità della clientela (turisti e residenti) e la volontà di non sovraccaricare cognitivamente l'utente ci hanno spinto a progettare il sito con un design minimale e pulito che non tendesse verso uno stile polarizzante. Si è scelto di puntare su un branding coerente con l'identità marittima: seppur il bianco sia predominante di modo da garantire leggibilità e chiarezza delle informazioni, è stata adottata una palette cromatica basata su diverse tonalità di blu per richiamare il tema nautico e, sfruttando la psicologia dei colori, trasmettere eleganza, calma, sicurezza e freschezza.

È stata poi mantenuta una rigida separazione tra struttura (HTML), presentazione (CSS) e comportamento (PHP e JavaScript). Sappiamo bene che mescolare questi elementi è il primo passo verso il debito tecnico e un debito formativo nel corso di Tecnologie Web, quindi abbiamo puntato sulla modularità per rispettare gli standard web attuali.

Infine, la progettazione del sito è stata condotta cercando di garantire l'accessibilità a tutte le categorie di utenti.

2.2 Struttura

La struttura del sito segue il modello gerarchico schematizzato in [Figura 1](#). In questa fase si è pianificata una suddivisione nelle seguenti pagine principali, accessibili tramite un menù di navigazione globale:

- **Home:** La pagina Home funge da punto di snodo principale. Deve contenere le informazioni essenziali di SailUP, utilizzando immagini di impatto per catturare l'attenzione del visitatore e offrire collegamenti rapidi alle funzionalità principali, quindi le sezioni Noleggio ed Esperienze. Di fondamentale importanza la chiarezza delle informazioni presentate, se l'utente si perde qui il resto del lavoro è inutile.
- **Cataloghi Noleggio ed Esperienze:** Queste pagine permettono all'utente di visualizzare l'offerta completa. Prevedono sistemi di filtraggio e ordinamento per agevolare la ricerca.

Selezionando un elemento, l'utente accede a una pagina di dettaglio dove può consultare le specifiche e procedere alla prenotazione. Il sistema effettuerà un controllo sulla disponibilità delle date scelte restituendo un feedback all'utente.

- **Blog:** La pagina Blog raccoglie articoli informativi e consigli turistici. Ogni articolo è visualizzabile singolarmente. Questa sezione è fondamentale per dare spessore al sito e trasformare una semplice piattaforma di noleggio in un riferimento per chi pianifica una giornata in mare.
- **Pagine Informative “Chi Siamo”, “FAQ”, “Privacy” e “Cookie”:** Tutto ciò che serve per dare credibilità al sito e per, potenzialmente, ridurre il carico di assistenza diretta. Le “FAQ” sono necessarie a rispondere ai dubbi più comuni prima che diventino telefonate o email, mentre le informative su “Privacy” e “Cookie” garantiscono che il sito non sia solo bello ma anche a norma. La pagina “Chi Siamo” presenta invece il team di SailUP.
- **Area Riservata:** Questa sezione gestisce l'accesso alla piattaforma. La pagina di **Login/Registrazione** permette all'utente di autenticarsi o creare un nuovo profilo. Una volta loggato, il sistema indirizza l'utente alla vista corretta in base al suo ruolo:
 - **Pagina Profilo Cliente:** Offre al cliente la possibilità di visualizzare e modificare i propri dati anagrafici. Include una sezione per consultare lo storico delle prenotazioni (attive e passate), permettendo all'utente di avere riscontro immediato sulle proprie attività.
 - **Dashboard Amministratore:** Questa sezione funge da centro di controllo. Permette di visualizzare la totalità delle prenotazioni nel sistema, accedere alla pagina profilo personale, gestire l'anagrafica degli utenti registrati e modificare dinamicamente i contenuti del sito (aggiunta/modifica/rimozione di Barche, Esperienze e Articoli del Blog).

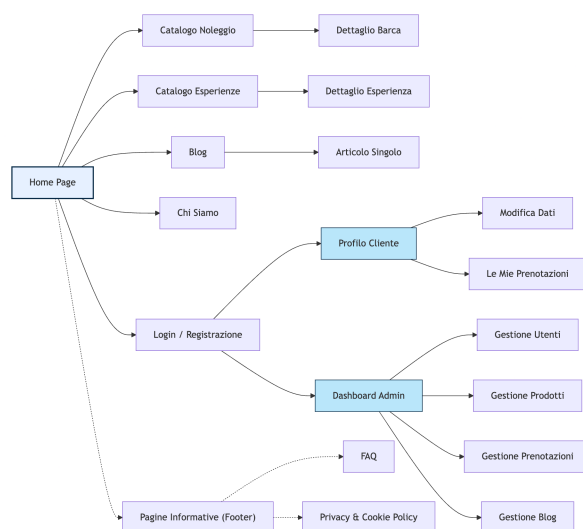


Figura 1: Mappa gerarchica della piattaforma SailUP

3 Realizzazione

In questa sezione abbandoniamo le astrazioni progettuali per approfondire i dettagli implementativi dello sviluppo di SailUP.

3.1 Front-End

3.1.1 Struttura (HTML)

La costruzione delle pagine web sfrutta il markup di HTML5, garantendo una chiara gerarchia delle informazioni. L'ossatura di ogni documento sfrutta i tag standard `<header>`, `<nav>`, `<main>` e `<footer>`, che permettono agli utenti di *screen reader* di orientarsi rapidamente all'interno della pagina.

- **Struttura generale:** La navigazione principale è contenuta nell' `<header>` e si adatta ai dispositivi mobili trasformandosi in un menu a scomparsa gestito tramite un pulsante ad "hamburger", il cui stato è comunicato alle tecnologie assistive tramite l'attributo `aria-expanded`.

Per facilitare l'esperienza d'uso via tastiera è stato inserito all'inizio del `<body>` il collegamento nascosto *Skip Link*, che consente di saltare i menù ripetitivi andando direttamente al contenuto principale della pagina.

Il corpo centrale della pagina è racchiuso nel tag `<main>`, al cui interno i contenuti sono organizzati logicamente: le schede del singolo prodotto nei cataloghi sono marcate con il tag `<article>`, identificandole come entità indipendenti, mentre le sezioni accessorie, come i filtri di ricerca e i riepiloghi d'ordine, sono delimitate dal tag `<aside>`.

- **Breadcrumbs:** L'orientamento all'interno delle pagine è agevolato dalle breadcrumbs, presenti in tutte le pagine ad eccezione di quelle d'errore.
- **Attributi e semantica:** Particolare attenzione è stata inoltre posta nel definire attributi adeguati per il contenuto e gli elementi funzionali: ai termini in lingua inglese è stato associato l'attributo `lang="en"` (es. *Privacy Policy*, *Login*), alle sigle ed acronimi (es. CAP, NA, S.r.l., FAQ) l'attributo `title` all'interno del tag `<abbr>` per esplicitarne il significato e alle date l'attributo `datetime` nel tag `<time>` per renderle *machine-readable* e quindi interpretabili da motori di ricerca e *screen reader*.
- **Ottimizzazioni:** Sono stati infine impiegati attributi specifici per migliorare l'esperienza utente, come `autocomplete` e `pattern` per facilitare la compilazione dei form e `loading="lazy"`, una soluzione tanto semplice quanto efficace per non uccidere le prestazioni del sito al primo caricamento.
- **Contenuti e asset multimediali:** La totalità dei testi e delle immagini presenti all'interno del sito è stata prodotta utilizzando AI Generativa, in particolare Gemini 3. Questa scelta ci ha consentito da un lato di sopperire alla nostra mancanza di competenze nel settore nautico, garantendo descrizioni e articoli tecnicamente corretti e verosimili, e dall'altro di popolare la piattaforma con materiale visivo di buona qualità e privo di vincoli di copyright. Per assicurare tempi di caricamento rapidi tutti gli asset così ottenuti sono stati poi convertiti nel formato .webp.

3.1.2 Presentazione (CSS)

La parte grafica è gestita interamente tramite fogli di stile CSS, mantenendo una netta separazione tra struttura e presentazione, una scelta che oltre a rispettare gli standard, ci ha risparmiato diverse emicranie in fase di revisione. Per gestire i ridimensionamenti intermedi (es. tablet), senza frammentare eccessivamente il codice in fogli diversi, abbiamo integrato

le media query direttamente nei file principali. Abbiamo preso questa scelta per evitare la proliferazione di file da poche decine di righe che complicherebbero la manutenzione e, contemporaneamente, per ottimizzare le prestazioni riducendo le richieste HTTP al server.

3.1.2.1 Style.css (Desktop e Base)

Questo foglio di stile definisce l'identità visiva principale del sito. Le scelte stilistiche includono:

- **Responsive design:** L'interfaccia adotta un approccio fluido che si adatta alle diverse risoluzioni dello schermo. Per il posizionamento degli elementi sono state impiegate le tecnologie **Flexbox** per header e footer e **CSS Grid** per le griglie dei prodotti e le specifiche tecniche.
- **Variabili:** L'uso di variabili CSS definite in `:root` ci ha permesso di centralizzare la gestione del tema. Questo facilita la manutenzione e abilita il supporto alla **Dark Mode** semplicemente modificando i valori delle variabili colore per la modalità scura.
- **Grid e flexbox:** L'impaginazione sfrutta CSS Grid per le strutture bidimensionali (come le card dei prodotti) e Flexbox per gli allineamenti monodimensionali (header e navbar).
- **Accessibilità visiva:** I colori scelti rispettano i criteri di contrasto WCAG AA. Inoltre, è stato definito un feedback visivo chiaro per gli stati di interazione (`:hover`, `:focus`), migliorando l'usabilità per chi naviga da tastiera.

3.1.2.2 Mobile.css (Dispositivi Portatili)

Richiamato tramite media query per dispositivi con larghezza inferiore a 768px, questo foglio di stile ottimizza l'esperienza utente su schermi ridotti:

- **Navigazione semplificata:** Il menu di navigazione orizzontale viene nascosto e viene introdotto un menù "Hamburger" espandibile, massimizzando lo spazio disponibile per i contenuti.
- **Linearizzazione del layout:** Le griglie multi-colonna, come le card per i prodotti, vengono riconfigurate in un layout a colonna singola per facilitare la lettura e lo scorrimento verticale.
- **Tabelle responsive:** Per risolvere il problema della leggibilità delle tabelle su schermi stretti, le righe vengono trasformate visivamente in "card". Si tratta di una soluzione necessaria per evitare che l'utente debba navigare i dati tramite frustranti scorrimenti orizzontali, un'esperienza inconciliabile con un servizio che promette relax.
- **Aree interattive:** Le dimensioni dei pulsanti e delle aree interattive sono aumentate per facilitare l'interazione tramite tocco.

3.1.2.3 Print.css (Stampa)

Per assicurare che i contenuti siano fruibili in maniera ottimale su carta è stato predisposto un foglio di stile dedicato, che modifica la struttura di una pagina come segue:

- **Rimozione degli elementi superflui:** Gli elementi interattivi inutili su carta (menu, breadcrumb, pulsanti "prenota", hero images) vengono nascosti tramite la classe `.print-none`, lasciando solamente il contenuto informativo essenziale. Il footer, ad esempio, viene sfoltito di tutti gli elementi superflui lasciando solamente dati utili come indirizzo e P.IVA.
- **Ottimizzazioni per la lettura:** Il font viene cambiato globalmente in *Times New Roman* (serif), più leggibile su supporto cartaceo rispetto ai font sans-serif usati a video. I colori

vengono forzati al nero su bianco e i link perdono la sottolineatura per una pulizia visiva maggiore.

- **Layout adattivo:** La struttura a colonne viene linearizzata, permettendo al contenuto principale di occupare l'intera larghezza del foglio stampato, evitando tagli laterali.
- **Gestione griglie:** Le sezioni a griglia vengono mantenute ma adattate con l'aggiunta di bordi per delimitare le aree, sostituendo la distinzione cromatica che viene persa in stampa.
- **Visualizzazione link esterni:** Sebbene attualmente disabilitata in assenza di collegamenti esterni, è stata predisposta una regola per stampare in chiaro l'URL di destinazione accanto ai link. Questo accorgimento permette a chi consulta la versione cartacea di conoscere l'indirizzo delle risorse citate, altrimenti irrecuperabile su carta.

3.1.3 Convenzioni interne

Oltre agli standard web generali, il progetto adotta specifiche convenzioni stilistiche e funzionali per garantire un'esperienza utente coerente e prevedibile:

- **Link:** i collegamenti ipertestuali sono distinguibili dal testo grazie alla sottolineatura presente. I link visitati, poi, assumono una colorazione azzurra, in linea con l'identità del brand.
- **Pagina corrente e link circolari:** Nei menu di navigazione la voce corrispondente alla pagina attuale è evidenziata visivamente con una sottolineatura blu e resa non cliccabile. In generale tutti i link che normalmente riporterebbero alla pagina corrente, i link circolari, vengono resi non cliccabili per evitare ricaricamenti inutili, un dettaglio tecnico utile per non confondere l'utente e risparmiare traffico inutile al server.
- **Convenzione font:** *Montserrat* è riservato esclusivamente alle intestazioni (h1-h6) e ai bottoni per impatto visivo, mentre *Open Sans* è utilizzato per tutto il corpo del testo.
- **Badge:** Nelle tabelle di riepilogo (es. prenotazioni), lo stato viene esplicitato da etichette colorate in base allo stato dell'elemento.
- **Divisione contenuti:** I contenuti indipendenti, come i prodotti nei cataloghi, sono sempre incapsulati in card con bordi arrotondati per facilitarne la distinzione.
- **Feedback nei Form:** I messaggi di aiuto sono sempre posizionati sotto il campo input in colore grigio, mentre i messaggi di errore appaiono in rosso.

3.1.4 Comportamento (JavaScript)

Le funzionalità interattive lato client sono gestite da script modulari che arricchiscono l'esperienza utente secondo il principio del **Progressive Enhancement**, un modo elegante per dire che il sito deve restare in piedi anche se l'utente decide di disabilitare gli script.

Il file `script.js` contiene le seguenti funzioni:

- **Menu mobile:** Gestisce l'apertura e chiusura del menu ad "hamburger", alternando le icone di stato (aperto/chiuso) e sincronizzando l'attributo ARIA `aria-expanded` per garantire la corretta comunicazione dello stato alle tecnologie assistive.
- **Modalità scura:** Controlla il cambio del tema visivo (chiaro/scuro) agendo sull'attributo `data-theme` del tag `html` e memorizzando la preferenza dell'utente nel `localStorage` per mantenere la scelta nelle visite successive.
- **Filtri:** Viene implementato un sistema di filtraggio per lo storico delle prenotazioni. Questo permette di visualizzare istantaneamente le prenotazioni in base al loro stato ("Tutte",

“Active”, “Complete”) agendo sulla visibilità delle righe della tabella e aggiornando in tempo reale i contatori presenti nelle tab di filtro.

- **Pulsante torna su:** Gestisce la comparsa del tasto per tornare ad inizio pagina dopo che l’utente scorre la pagina, gestendo attraverso `tabindex` l’attivazione della navigazione da tastiera sul pulsante qualora diventi visibile.
- **Toggle password:** Gestisce la visualizzazione in chiaro dei campi password, una piccola cortesia per evitare che l’utente debba digitare tre volte una stringa complessa a causa di un errore di battitura.
- **Persistenza dei dati:** Le date selezionate nei cataloghi vengono riportate automaticamente nel form di prenotazione.
- **Calcolo dinamico dei prezzi:** Nei form di prenotazione ogni selezione che modifica il prezzo, come il cambiamento delle date o l’aggiunta di extra, viene monitorata e il totale viene istantaneamente aggiornato in modo da dare sempre contezza all’utente di quanto andrà a pagare se decide di proseguire con la prenotazione.
- **Gestione del focus:** In caso di messaggi dal server o errori di validazione lo script forza lo scroll della pagina verso il messaggio e vi sposta il focus, assicurandosi che anche un utente che utilizza uno *screen reader* (o un utente distratto) non manchi l’avviso.

I file di validazione (`register_validation.js`, `login_validation.js`, `blog_validation.js`, `product_validation.js`, `payment_validation.js`) garantiscono l’integrità dei dati e migliorano l’usabilità dei form:

- **Validazione in tempo reale:** Verifica la correttezza del campo alla perdita del focus, controllando formati complessi come il Codice Fiscale, la validità strutturale delle Email o delle URL.
- **Assistenza all’input:** Include comportamenti come la conversione automatica in maiuscolo dei caratteri durante la digitazione nei campi Codice Fiscale e Provincia.
- **Validazione dei pagamenti:** Gestisce la formattazione automatica del numero di carta (gruppi di 4 cifre) e della scadenza (MM/AA), oltre a verificare che il CVV non contenga lettere o caratteri speciali.
- **Invio del modulo:** Lo script intercetta il tentativo di invio del modulo e lo valida. Se la validazione fallisce la richiesta al server viene bloccata e la pagina esegue uno scroll automatico verso il primo campo errato, portandovi il focus per facilitare la correzione immediata.

3.2 Back-End

3.2.1 Architettura (PHP)

Lo sviluppo lato server è stato realizzato con un approccio modulare che simula il pattern architetturale **Model-View-Controller** (MVC), garantendo una chiara separazione delle responsabilità e facilitando la manutenzione del codice. La struttura è la seguente:

- **Model:** La classe `DBConnection` (file `db_connection.php`) incapsula tutta la logica di accesso ai dati. Questa classe centralizza le query al database e implementa metodi specializzati per ogni operazione CRUD (Create, Read, Update, Delete) utilizzando esclusivamente **Prepared Statements**, garantendo sicurezza nativa contro SQL injection.

- **Controller:** Ogni pagina PHP nella directory `public/php/` agisce da controller dedicato. Il controller gestisce il flusso dell'applicazione: verifica i permessi dell'utente tramite le funzioni di sessione, elabora i dati ricevuti via POST/GET, interagisce con il Model per recuperare o modificare i dati, e infine prepara le variabili necessarie per la vista.
- **View:** I template HTML statici nella directory `public/pages/` costituiscono le viste. La funzione `buildPage()` carica questi template e inietta dinamicamente header, footer e contenuti tramite un sistema di segnaposto (es. `[HEADER]` , `[FOOTER]`). Questo approccio garantisce la separazione tra logica e presentazione, mantenendo il markup pulito e facilmente modificabile.

3.2.1.1 Modularità del codice

Il file `pages.php` definisce un array associativo che mappa ogni identificatore di pagina al suo filename PHP corrispondente. Questo sistema elimina la duplicazione di URL hard-coded nel codice, facilita la manutenzione (modificare un URL richiede la modifica di un solo punto) e permette la generazione dinamica dei menu di navigazione.

All'interno della directory `includes/` , inoltre, sono presenti i seguenti moduli funzionali:

- **helpers.php:** Contiene le funzioni di utilità per la generazione dinamica delle componenti principali della pagina come header, footer e card relative ai prodotti salvati in database. Le funzioni `buildHeader()` e `buildFooter()` gestiscono automaticamente lo stato dei link (pagina corrente evidenziata e non cliccabile), l'aggiunta di attributi ARIA per l'accessibilità, e la visualizzazione condizionale degli elementi in base allo stato di autenticazione dell'utente.
- **session/session.php:** Gestisce lo stato della sessione e fornisce funzioni di controllo accesso, lasciandoci abbastanza sereni che l'utente non si risvegli improvvisamente con i privilegi di un amministratore.
- **auth/auth.php:** Centralizza le operazioni di registrazione e login. La funzione `registerUserFull()` orchestra l'inserimento coordinato di utente e indirizzo in una transazione logica, mentre `loginUserAuth()` gestisce l'autenticazione verificando le credenziali con `password_verify()` .
- **utils/validation.php:** Contiene le funzioni di validazione lato server per tutti i tipi di input (es. password, codice fiscale). Queste validazioni rafforzano quelle client-side, implementando il principio della **doppia validazione** come best practice di sicurezza perché, per quanto l'utente possa sembrare inoffensivo, la fiducia nel client è una pratica su cui non vogliamo far affidamento.

3.2.2 Logica di Business

La logica delle prenotazioni rappresenta uno dei componenti più critici del sistema. Il metodo `checkDateAvailability()` implementa un algoritmo di verifica della disponibilità che controlla tutte le prenotazioni esistenti per il prodotto specificato (barca o esperienza), verificando la sovrapposizione temporale tra la nuova richiesta e le prenotazioni attive usando tre condizioni logiche che coprono tutti i possibili casi di overlap, un labirinto di logica booleana necessario ad evitare che la gestione del calendario si trasformi in un'anarchia totale.

Questa verifica viene eseguita sempre prima di confermare una prenotazione, garantendo che non si verifichino doppie prenotazioni dello stesso prodotto per periodi sovrapposti.

Il sistema implementa il pattern **Post-Redirect-Get** per tutte le operazioni che modificano lo stato (registrazioni, login, creazione contenuti). Dopo aver processato una richiesta POST, il server:

1. Elabora i dati e effettua le modifiche necessarie
2. Memorizza eventuali messaggi di successo/errore in sessione
3. Esegue un redirect HTTP (header `Location:`) verso una pagina di visualizzazione
4. La pagina di destinazione mostra il messaggio recuperandolo dalla sessione

Questo approccio previene la ri-sottomissione accidentale del form (tramite refresh o navigazione back), migliora l'esperienza utente e rende il flusso dell'applicazione più robusto e prevedibile, evitandoci di dover gestire dati duplicati.

3.2.2.1 Sistema CRUD Amministrativo

Il pannello amministrativo implementa operazioni complete di Create, Read, Update e Delete per tutte le entità dinamiche:

- **Gestione Prodotti (Barche/Esperienze):** L'amministratore può inserire nuovi prodotti specificando tutte le caratteristiche (nome, tipo, descrizione, prezzo, capacità), gestire i servizi extra a pagamento, i servizi inclusi nel prezzo, e le lingue disponibili per le esperienze guidate. Il sistema supporta l'upload di immagini che vengono memorizzate nella directory `public/img/prodotti/` e referenziate nel database tramite URL.
- **Gestione Articoli Blog:** Ogni articolo supporta contenuti strutturati attraverso la tabella `Articolo_Blog_Extra`, che permette di inserire sezioni diverse (paragrafi, liste puntate, consigli) mantenendo l'ordine di visualizzazione. Questo approccio rende gli articoli flessibili e facilmente estensibili.
- **Gestione Utenti e Prenotazioni:** L'amministratore ha visibilità completa su tutti gli utenti registrati e può monitorare lo stato di tutte le prenotazioni nel sistema, facilitando la gestione operativa dell'attività.

3.2.2.2 Gestione Race Condition

Per prevenire la **race condition** che potrebbe verificarsi quando più utenti tentano di prenotare lo stesso prodotto contemporaneamente è stato implementato il metodo `verificaDisponibilitaProdotto()`, che viene eseguito immediatamente prima dell'inserimento della prenotazione nel database e si occupa di effettuare un controllo atomico della disponibilità, verificando l'esistenza di prenotazioni attive che si sovrappongano temporalmente con il periodo richiesto.

Questo controllo è applicato in tre punti critici del flusso di prenotazione:

- In `pagamento.php`, immediatamente prima di confermare il pagamento con carta di credito
- In `dettaglio_barca.php` e `dettaglio_esperienza.php`, prima di creare prenotazioni con pagamento in contanti o bonifico

Se il prodotto non è più disponibile l'utente riceve un messaggio che chiarisce il problema ed invita l'utente a selezionare una data alternativa, offrendo un'esperienza utente chiara anche in situazioni di alta concorrenza.

3.2.2.3 Validazione

Il sistema implementa il principio della **doppia validazione** come best practice di sicurezza, controllando ogni input non solo lato client (JavaScript) ma anche lato server (PHP). La validazione *server-side* rappresenta la principale barriera di sicurezza, garantendo che anche in caso di JavaScript disabilitato, o di richieste manipolate da utenti un po' troppo curiosi, il server rifiuti sempre dati non conformi.

Ogni dato ricevuto via POST request viene ricontrollato.

3.2.3 Sessioni, Sicurezza e Protezione dei Dati

Il mantenimento dello stato utente è gestito tramite un sistema dedicato nel file `session.php`. All'avvio lo script verifica che la sessione non sia già attiva evitando così errori di sessioni duplicate. Sono state poi implementate funzioni helper per semplificare i controlli di accesso trasversali:

- `isLoggedIn()` : Verifica la presenza della chiave `user` nella sessione, indicando un utente autenticato.
- `isAdmin()` : Controlla se l'utente corrente possiede il flag di amministratore.
- `requireLogin()` : Posta all'inizio delle pagine protette, interrompe l'esecuzione e reindirizza alla pagina di login se l'utente non è autenticato.
- `requireGuest()` : Funzione complementare che reindirizza gli utenti già loggati, utile per pagine come login e registrazione.
- `requireAdmin()` : Blocca l'accesso alle pagine amministrative agli utenti non privilegiati, mostrando una pagina di errore 403.

Questo approccio permette di proteggere le risorse sensibili con una singola riga di codice all'inizio di ogni controller, mantenendo il sistema sicuro e il codice leggibile.

3.2.4 Sicurezza lato Server

La sicurezza è stata considerata prioritaria in ogni fase dello sviluppo backend. Oltre alla validazione degli input, sono state implementate le seguenti misure di protezione:

- **Prevenzione SQL injection:** Tutti i metodi della classe `DBConnection` utilizzano esclusivamente Prepared Statements. I parametri vengono vincolati alla query tramite `bind_param()` e mai concatenati direttamente nelle stringhe SQL, eliminando alla radice il rischio di iniezione di codice malevolo. Questo approccio è applicato uniformemente a tutte le query, dalle più semplici SELECT alle operazioni complesse di JOIN multi-tabella.
- **Hashing delle password:** Le password non vengono mai salvate in chiaro nel database. In fase di registrazione, viene utilizzata la funzione `password_hash()` con l'algoritmo `PASSWORD_DEFAULT` (attualmente Bcrypt), che genera automaticamente un salt casuale e produce un hash sicuro. Al login, la verifica avviene tramite `password_verify()`, che confronta la password fornita con l'hash memorizzato in modo sicuro. Questo garantisce la

protezione delle credenziali anche in caso di compromissione del database, di modo che in caso di *leak* un malintenzionato possa recuperare qualcosa di utile solo sostenendo lunghe sessioni di *brute force*.

- **Prevenzione Cross-site scripting:** Ogni dato dinamico stampato nell'HTML viene sanitizzato tramite `htmlspecialchars()`, che converte i caratteri speciali (come `<`, `>`, `"`, `'`, &) in entità HTML sicure. Questo impedisce l'esecuzione di script JavaScript malevoli iniettati attraverso input utente, proteggendo sia gli utenti che il sistema da attacchi XSS.
- **Prevenzione Cross-site request forgery:** ogni form che modifica lo stato del sistema (login, registrazione, creazione/modifica prodotti) è protetto da un token CSRF, generato e memorizzato in sessione. La funzione `verifyCsrfToken()` valida il token ricevuto confrontandolo con quello in sessione usando `hash_equals()`, una funzione timing-safe che previene attacchi di tipo timing. Il token viene iniettato come campo nascosto in ogni form e validato nel controller prima di processare qualsiasi operazione, garantendo che la richiesta provenga effettivamente dal sito e non da una fonte malevola.
- **Gestione sicura delle sessioni:** La sessione viene avviata con controllo preventivo tramite `session_status()` per evitare errori. I dati sensibili in sessione sono ridotti al minimo necessario, e il logout effettua una pulizia completa della sessione con `session_destroy()`.
- **Configurazione externalizzata:** Le credenziali del database e altre informazioni sensibili sono definite nel file `conf.php`, che è escluso dal sistema di versionamento Git tramite `.gitignore`. Questo previene l'esposizione accidentale di credenziali in repository pubblici.
- **Prepared statements:** L'interazione con il database si serve di metodi che utilizzano esclusivamente *Prepared Statements* con parametri vincolati. Questo approccio elimina il rischio di SQL injection impedendo l'inserimento di codice SQL malevolo.
- **Nota sul deployment - Permessi di upload:** In fase di deployment sul server universitario è stata riscontrata un'incongruenza tra l'utente proprietario dei file (studente) e l'utente esecutore del processo Apache. Non potendo intervenire sui privilegi di sistema per allineare l'ownership della cartella di upload, si è optato per l'impostazione dei permessi a 777 limitatamente alle directory di destinazione delle immagini. Siamo consapevoli che questa configurazione non sia ottimale dal punto di vista della sicurezza, in quanto consente a qualsiasi processo sul server di leggere, scrivere ed eseguire file in tali cartelle. Tuttavia, in assenza di accesso ai privilegi di amministratore (sudoers) per modificare l'ownership o configurare gruppi condivisi, questa soluzione rappresenta l'unico compromesso praticabile per garantire il corretto funzionamento della funzionalità di upload nel contesto dell'ambiente di hosting fornito.

3.2.5 Database (SQL)

L'interazione con il database è completamente incapsulata nella classe `DBConnection`. Questa architettura offre le seguenti funzionalità:

- **Gestione della connessione:** La connessione al database segue il pattern di apertura/chiusura esplicita. `openConnection()` apre la connessione solo quando necessario, imposta il charset UTF-8 per supportare caratteri internazionali, di modo da non dover vedere

trasformare una “è” in un rombo con il punto interrogativo, e gestisce errori di connessione in modo sicuro. `closeConnection()` chiude la connessione al termine di ogni operazione, liberando risorse e prevenendo connection leak.

- **Prepared statements:** oltre alla funzione relativa alla sicurezza citata in precedenza, i *Prepared Statements* separano la struttura della query dai dati e migliorano le performance grazie al piano di esecuzione pre-compilato dal database.
- **Gestione robusta degli errori:** La classe implementa una gestione degli errori attraverso codici di ritorno specifici dove necessario. Questo sistema permette al controller di fornire feedback precisi all’utente (E.g.: “Email già registrata”) senza esporre dettagli tecnici del database.

3.2.5.1 Entità del Database

Le entità implementate nel database sono:

- **Utente:** Contiene le informazioni di tutti gli iscritti, come credenziali e dati anagrafici. Il campo `Is_Admin` serve per definire i privilegi di accesso.
- **Indirizzo:** Memorizza i dati geografici (Via, Città, CAP) separandoli dall’utente per una migliore normalizzazione.
- **Prodotto:** Contiene i dati comuni dei prototipi, Noleggi ed Esperienze, del catalogo (prezzo, descrizione, posti).
- **Prodotto_Extra:** Gestisce i servizi opzionali a pagamento (es. Skipper, Champagne).
- **Prodotto_Incluso:** Elenca i servizi già compresi nel prezzo base (es. Carburante, Assicurazione).
- **Lingua:** Memorizza Le lingue supportate per i servizi guidati (IT, EN, FR, ES).
- **Prodotto_Lingua:** Relazione che collega Prodotto e Lingua, indicando quali lingue sono parlate nell’esperienza specifica.
- **Prenotazione:** Necessaria per mantenere lo storico delle transazioni.
- **Articolo_Blog:** Necessaria per gestire gli articoli del blog.
- **Articolo_Blog_Extra:** Struttura i contenuti complessi degli articoli (es. liste puntate, sezioni “Cosa portare”).
- **Media:** centralizza la gestione delle immagini.

4 Accessibilità e Testing

L’accessibilità è stata un pilastro del progetto, guidata dai principi studiati durante il corso e dalle linee guida internazionali. L’obiettivo è stato quello di garantire la fruizione dei contenuti e delle funzionalità, per quanto possibile, a tutte le categorie di utenti indipendentemente da eventuali disabilità fisiche, cognitive o limitazioni tecnologiche, in conformità con le linee guida WCAG e i principi PURO (Percepibile, Utilizzabile, Comprensibile, Robusto). Alla fase di sviluppo è seguita poi quella di validazione e *testing*, un vero e proprio bagno di umiltà che ci ha permesso di individuare tutta quella moltitudine di errori e mancanze sfuggiteci.

4.1 Accessibilità

Il garantire l'accessibilità del sito a tutte le categorie di utente ha richiesto accorgimenti su tutte le componenti del progetto: struttura, presentazione e comportamento. Vengono elencati di seguito tutte le attenzioni riposte, al netto di inevitabili dimenticanze:

- **Principi WCAG e struttura semantica:** Abbiamo utilizzato con rigore i tag semantici HTML (`<main>` , `<nav>` , `<header>` , `<footer>`) per fornire una mappa logica immediata del sito, facilitando l'interpretazione dei contenuti da parte delle tecnologie assistive come gli *screen reader*.
- **Navigazione da tastiera:** Il sito è stato progettato per essere completamente navigabile utilizzando esclusivamente la tastiera. Attraverso l'estensione *Wave* è stato verificato per ogni pagina che l'ordine di focus mediante tabulazione avvenisse correttamente.
- **Salta al contenuto:** È stato implementato il link “Salta al contenuto” per permettere agli utenti di *screen reader* di saltare i blocchi di navigazione ripetitivi e a loro superflui.
- **Dettagli semantici:** Abbiamo riposto attenzione a quegli attributi che rimangono invisibili agli utenti comuni ma che sono fondamentali per *ranking* e per la corretta sintesi vocale attraverso *screen reader*. L'uso dell'attributo `lang` (principalmente per termini in inglese e in francese) evita una riproduzione maccheronica della sintesi vocale, mentre il tag `<abbr>` e l'attributo `datetime` rendono acronimi e date *machine-readable*.
- **Semantica dinamica:** Gran parte dei contenuti di SailUP è dinamica. Per evitare di inserire manualmente i tag di accessibilità ad ogni occorrenza, abbiamo implementato nel modulo `helpers.php` una serie di funzioni di formattazione che, consultando dei dizionari presenti nella directory `/config` , automatizzano l'inserimento di attributi e tag. `formatTextAbbr` si occupa di trasformare automaticamente acronimi (es. “GPS”, “TV”) ed unità di misura (es. “m”, “h”, “cv”) nel tag `abbr` con il relativo title esplicativo, mentre `formatTextLang` si occupa di identificare i termini stranieri (es. “skipper”, “champagne”) assegnandogli il corretto attributo `lang` .
- **Alternative testuali:** Ogni immagine che fornisce informazioni aggiuntive rispetto al contesto, ovvero non puramente decorativa, è stata dotata di un attributo `alt` descrittivo. Per le immagini puramente grafiche tale attributo è stato lasciato vuoto anziché rimosso per permette allo *screen reader* di interpretare la scelta come intenzionale e non come una svista in fase di sviluppo. I form amministrativi che consentono l'aggiunta di prodotti e articoli del blog includono inoltre un campo opzionale che permette all'occorrenza di riempire questo campo.
- **Contrasto cromatico e colori:** È stata prestata particolare attenzione ad utilizzare una palette cromatica che mantenesse i rapporti cromatici tali da rispettare almeno il livello AA delle WCAG 2.2 pur rimanendo esteticamente gradevole, operazione che ci è costata più tempo di quanto vorremmo ammettere. L'implementazione di un selettore di tema `light/dark` offre inoltre agli utenti la possibilità di scegliere la modalità di visualizzazione che preferiscono, migliorando ulteriormente la leggibilità. Sebbene il tema testato e validato per rispettare il livello AA delle linee guida WCAG sia quello chiaro, abbiamo riposto attenzione anche per quello scuro.

- **WAI-ARIA:** Poiché gran parte di SailUP vive di interazioni in tempo reale, abbiamo sfruttato gli attributi WAI-ARIA per evitare che l'esperienza d'uso si trasformasse in un silenzio assordante per chi usa uno *screen reader*. Abbiamo utilizzato `aria-live="polite"` e i ruoli `status/alert` per fare in modo che venissero comunicati i campi obbligatori e i messaggi di feedback. Attraverso l'uso di `aria-describedby` abbiamo collegato ogni campo di input alle proprie istruzioni e ai messaggi d'errore specifici. Nella navigazione abbiamo sfruttato `aria-current="page"` e la coppia `aria-expanded/aria-controls` per comunicare dinamicamente lo stato del menu mobile. Abbiamo cercato di rendere l'interfaccia meno generica delegando al backend la generazione di `aria-label` descrittivi, ad esempio nelle card dei prodotti un link “Dettagli” avrà associato il nome del prodotto. Per pulire il flusso audio da rumore inutile, infine, abbiamo sfruttato `aria-hidden="true"` per evitare che icone puramente decorative ed emoji venissero lette. Qualora queste icone venissero inserite attraverso foglio di stile CSS abbiamo provveduto a sostituirle con immagini.

4.2 Validazione e Testing

Il codice è stato esaminato per individuare quegli errori che, inevitabilmente, passano inosservati durante la fase di sviluppo. Non possiamo nascondere quanto ottenere un “bollino verde” da un validator induca in noi un rilascio istantaneo di dopamina.

4.2.1 Validazione

- **Validatori W3C HTML e CSS:** Sono stati utilizzati per assicurare che la struttura HTML5 e i fogli di stile siano conformi agli standard internazionali. Al netto di qualche avviso relativo all'uso delle variabili CSS (ces. `var(--colore-primario)`), il codice ha superato i test senza errori critici.
- **Total Validator:** Abbiamo utilizzato questo strumento per una verifica più completa, testando contemporaneamente la validità dell'HTML, la conformità alle linee guida WCAG e l'integrità dei collegamenti ipertestuali. Se da un lato ci ha permesso di scovare diverse sviste sugli attributi ARIA, dall'altro abbiamo dovuto ignorare molti falsi positivi riguardanti lo spell-check. Il test delle pagine private, accessibili solo in seguito a login, sono stati eseguiti inviando il codice sorgente perchè altrimenti inaccessibili. Alcuni di questi controlli sono stati effettuati anche con “axe DevTools” per conferma.
- **WCAG Contrast Checker:** Questo strumento è stato prezioso per analizzare e correggere i contrasti tra gli elementi presenti all'interno delle pagine. Questo *tool* ci ha messo di fronte alla dura realtà che quel blu che ci piaceva tanto, purtroppo, non è per tutti facilmente leggibile.
- **w3ba11y:** Abbiamo utilizzato questo strumento per verificare che le *keywords* inserite fossero effettivamente presenti nella pagina.
- **WAVE e SilkTide:** Oltre a condurre un secondo controllo sui contrasti, abbiamo sfruttato questi *tool* per verificare che l'ordine di navigazione da tastiera fosse corretto. Oltre a questo ci hanno permesso di individuare altri errori relativi alle intestazioni. *WAVE* ha segnalato alcuni warning per “*redundant link*”, riferendosi al doppio collegamento alla *Home* presente sia sul logo che nel menu. In questo caso abbiamo esercitato il nostro diritto di libero arbitrio

ignorandolo: rimuovere uno dei due sarebbe stato tecnicamente “pulito” secondo il *tool*, ma poco intuitivo per un utente reale.

4.2.2 Test

Sono stati condotti test approfonditi sulla validazione degli input utente per garantire la robustezza e la sicurezza dei form. Ecco il resoconto delle principali manovre di verifica effettuate:

- **Form di registrazione:** Abbiamo provato ad immettere nei campi del form di registrazione input errati per verificare che i controlli inseriti funzionassero, come ad esempio nome e cognome di minimo 2 caratteri, codice fiscale di 16 caratteri alfanumerici, email in un formato valido (utente@dominio.it), password di minimo 8 caratteri contenente almeno una lettera maiuscola una minuscola e un numero ed accettazione obbligatoria per la Privacy Policy.
- **Form creazione prodotto e blog (Admin):** È stata verificata la validazione di tutti i campi obbligatori per garantire la completezza dei dati. Per i prodotti, i campi validati includono nome, tipo, descrizione, prezzo, capacità, URL immagine e testo alternativo associato. Per gli articoli del blog, i controlli si applicano a titolo, categoria, data, estratto, contenuto, URL immagine e testo alternativo.
- **Controllo degli accessi:** È stata verificata l'efficacia del sistema di controllo accessi tentando di forzare l'URL verso pagine protette (es. la dashboard amministrativa o l'area personale di un altro utente) partendo da uno stato di utente non autenticato o privo di privilegi. Sono stati inoltre testati i percorsi verso risorse inesistenti per verificare la corretta gestione delle pagine di errore.
- **Verifica della disponibilità:** La logica di prenotazione è stata testata simulando diverse richieste di noleggio per lo stesso prodotto, effettuando ad esempio prove di prenotazione in date con sovrapposizioni rispetto a prenotazioni già confermate.
- **Tecnologie assistive:** La piattaforma è stata navigata utilizzando screen reader quali VoiceOver e NVDA per assicurarsi che la navigazione fosse comprensibile anche attraverso l'uso di questi strumenti di sintesi vocale. È stato ad esempio controllato che gli elementi interattivi venissero correttamente etichettati, che i cambiamenti di elementi a schermo venissero comunicati o che elementi non necessari (es. emoji) non venissero letti. Tutto ciò ci ha ricordato che il codice perfetto non esiste, specialmente se deve essere interpretato da uno *screen reader*.
- **Test automatizzati di accessibilità:** Per verificare sistematicamente l'accessibilità di tutte le pagine PHP del sito è stato utilizzato lo strumento *pa11y* per l'esecuzione di test automatizzati. Questo *tool* ci ha permesso di analizzare l'intero sito in modo efficiente, identificando eventuali problemi di accessibilità su tutte le pagine dinamiche.
- **Audit automatizzato con Unlighthouse:** Per ottenere una valutazione completa delle performance, dell'accessibilità, delle best practices e della SEO del sito, è stato utilizzato Unlighthouse v0.17.4. I risultati ottenuti sono stati particolarmente soddisfacenti: **100% su SEO, 100% su Best Practices, 100% su Accessibility e 67% su Performance**. Il punteggio

relativo alle performance, seppur non perfetto, è comunque accettabile considerando la natura dinamica del sito, i limiti dell'hosting e la quantità di contenuti multimediali presenti.

- **Compatibilità e design responsivo:** Il sito è stato testato sui principali *browser* web moderni, tra cui Google Chrome, Opera, Mozilla Firefox, Safari e Microsoft Edge per accertarci che venissero renderizzati correttamente. Il design responsivo è stato verificato a diverse risoluzioni simulando dispositivi che vanno da smartphone ai tablet, fino ai monitor desktop. Non sono state invece prese in considerazione versioni più vecchie dei *browser*, considerato il profilo dell'utenza target del sito orientata all'utilizzo di dispositivi moderni.

5 Suddivisione del Lavoro

L'evoluzione del progetto è stata meno lineare del previsto. Inizialmente composto da un gruppo di quattro persone, il team è andato incontro ad una riorganizzazione procedendo ad un fork del progetto per ultimare il lavoro in tre.

Questa transizione ha reso la suddivisione dei compiti non definibile in maniera netta. Parti del codice non direttamente scritte dagli attuali membri del gruppo sono state oggetto di correzioni e miglioramenti. Proveremo di seguito a definire nel miglior modo possibile, memoria assistendoci, la suddivisione del lavoro.

- **Davide Biasuzzi:** Progettazione e creazione del database (SQL), pagina `db_connection.php` (PHP) per la gestione della connessione al database e delle query CRUD, implementazione della logica di business per la gestione delle prenotazioni (PHP), backend della parte non amministrativa del sito, revisione design pagine catalogo e di dettaglio dei prodotti, parte frontend della pagine di errore, scrittura della relazione parte backend.
- **Francesco Marcon:** Sviluppo frontend delle principali pagine (HTML), definizione stile mobile e desktop (CSS), validazione client-side per login, registrazione, pagamenti, inserimento prodotti e blog (JS), correzione di funzioni per il popolamento dinamico da DB, correzione errori grafici e adeguamento standard WCAG (PHP), scrittura relazione parte frontend.
- **Alberto Reginato:** Parte frontend delle principali pagine del sito (HTML, CSS), foglio di stile per la stampa (CSS), aggiunta bottone "Torna su" e calcolo prezzi in tempo reale (JS), funzioni per la generazione dinamica dei tag e attributi html (PHP), popolamento delle pagine e del DB, testing e validazione, scrittura della relazione parte frontend.
- **Hossam Ezzemouri:** Parte backend della parte amministrativa del sito (PHP), implementazione del sistema CRUD per la gestione di prodotti e articoli del blog, funzioni di autenticazione e gestione sessioni (PHP), revisione design pagine create e update di prodotti e blog.
 - **Correzioni effettuate dal team sulla parte prima del fork:**
 - Collegati pulsanti non funzionanti
 - Prevenzione traversal-path a login per utente già loggato
 - Aggiunta funzionalità di elimina utente per l'admin
 - Aggiunta funzionalità di eliminazione prodotto
 - Corretta logica breadcrumbs per la parte admin->profilo admin

- Aggiunta funzionalità profilo utente per l'admin
 - Revisione funzione di reperimento immagini dal DB,
 - Revisione politica di accesso alle pagine admin per utenti non admin
 - Varie migliorie grafiche minori
 - Revisione della validazione dati utente (indirizzo, nome e cognome)
 - Revisione sintassi HTML5 e accessibilità generale
 - Esplicitati e disambiguati errori per gli utenti generici
 - Implementata funzione elimina account utente
 - Validazione registrazione
- **Correzioni effettuate dal team sulla parte dopo il fork:**
- Rimossa paginazione mal funzionante nella sezione amministrativa
 - Revisione funzionalità create/update di blog
 - Revisione funzionalità create/update di prodotti
 - Revisione separazione tra contenuto e comportamento js
 - Revisionata validazione dei form lato client (errori su REGEX e messaggi)
 - Rimozione classi CSS inesistenti
 - Revisionata logica pulsanti su admin prenotazioni
 - Revisione su parte legata alla visualizzazione delle psw nel login

6 Conclusioni e Sviluppi Futuri

Messi di fronte alla complessità delle sfide tecniche (ed interpersonali) che a volte si sono rivelate più difficili del previsto, lavorare a questo progetto è stata un'esperienza estremamente formativa. SailUP non è probabilmente l'innovazione che cambierà le sorti del turismo globale ma è un lavoro di cui siamo orgogliosi e che saremmo fieri di esporre in un possibile portfolio personale. Ci ha dato modo di dimostrare la nostra capacità di costruire un'applicazione web che non sia solo funzionante ma anche solida, sicura e navigabile da tutti.

6.1 Possibili Sviluppi Futuri

La base tecnologica di SailUP è stata pensata per essere modulare quindi potenzialmente, se non avessimo altri esami da preparare, espandibile. Tra le evoluzioni possibili ipotizziamo:

- **Sistema di recensioni e valutazioni:** Aggiungere una funzionalità che permetta agli utenti di lasciare recensioni e valutazioni sui prodotti, aumentando la fiducia e fornendo un feedback prezioso.
- **Supporto contenuti video:** La tabella `Media` del database è già predisposta per gestire video oltre alle immagini (campo `Tipo_Media` con valori `Immagine` e `Video`). Sarebbe possibile implementare l'upload e la visualizzazione di video promozionali per barche ed esperienze, tour virtuali e contenuti video per il blog, arricchendo significativamente l'esperienza utente.
- **Notifiche automatiche via email:** Sviluppare un sistema per l'invio automatico di email di conferma, promemoria e aggiornamenti relativi alle prenotazioni effettuate dagli utenti.
- **Localizzazione:** SailUP parla attualmente italiano e un po' di inglese per i sintetizzatori vocali, ma il mercato turistico potrebbe richiedere un supporto multilingue completo.